

RescueRoute

Surplus Food Matching & Pickup Planner — Group Project Brief

Project purpose. Build an explainable Python application that helps coordinate time-sensitive surplus food donations with compatible community organizations and volunteer drivers. Your project should make clear, testable decisions—not simply produce a schedule.

Team size: 3 students Project duration: 3 weeks Submission: GitHub repository, documentation, demo, and final presentation

1. Project Overview

Restaurants, grocery stores, campus events, and caterers may have food that is safe to donate but must be picked up quickly. At the same time, shelters, food pantries, community fridges, and other organizations have different capacity limits, operating hours, and food-acceptance rules. RescueRoute is an educational prototype that matches donations to feasible recipients and volunteer drivers.

Your system should answer this question:

How can we prioritize time-sensitive food donations and assign them to compatible recipients and volunteers in a way that is efficient, transparent, and easy to explain?

This is not a production food-safety or routing system. Do not claim that your application is approved for real food-handling, medical, or emergency operations. Treat all data as simulated unless your group clearly documents a public, non-sensitive source.

2. What You Will Build

Each group will build a command-line Python application or a small web/dashboard application that:

- Loads donation, recipient, and volunteer information from CSV or JSON files.
- Validates IDs, quantities, food types, capacities, availability, and time values.
- Identifies feasible donation-recipient-volunteer matches.
- Implements a FIFO baseline and one clearly defined greedy strategy.
- Produces an explainable assignment or rejection message for every donation.
- Calculates metrics that compare the two strategies.
- Includes automated tests for normal and edge cases.

3. Data Model

Your exact fields may differ, but your data model must include the information needed to make and explain matching decisions.

Entity	Minimum fields
Donation	donation_id, donor name, food type, quantity, expiry/remaining shelf life, pickup area, ready time
Recipient	recipient_id, organization name, accepted food types, capacity, area/location, closing time
Volunteer	volunteer_id, vehicle capacity, area/location, availability, maximum pickups
Assignment	donation_id, recipient_id, volunteer_id, scheduled time, decision reason, status

Example design choice: Store donations and recipients in dictionaries keyed by ID for direct lookup. Use sets for accepted food types and duplicate-ID detection. Use a list for assignment history. Use a heap or sorted list only if it genuinely supports your greedy-priority rule.

4. Required Algorithms

Your project must compare two approaches on the same input data. The comparison is part of the assignment—not an optional feature.

Approach	Required behavior
A. FIFO baseline	Process donations in arrival order. For each donation, assign the first feasible recipient and volunteer. Record unmatched donations.
B. Greedy urgency-first strategy	Prioritize donations using an explicit rule. At minimum, earlier expiration must be considered. Your group may add quantity, recipient closing time, distance/area, or arrival time as tie-breakers.

Your README must state the greedy rule precisely. For example:

Greedy rule example: First schedule donations with the least remaining shelf life. If two donations have equal remaining shelf life, schedule the larger quantity first. If still tied, choose the donor with the earlier arrival time.

You are not required to prove that your greedy strategy finds a globally optimal schedule. You are required to explain what it optimizes locally, why it is reasonable, and where it may fail or create trade-offs.

5. Explainability Requirement

Every assignment and every unassigned donation must have a human-readable explanation. Do not output only IDs or a final table.

Outcome	Example explanation
Assigned	Donation D-1001 was assigned to Community Shelter with volunteer V-301 because it expires in 90 minutes, the shelter accepts prepared meals, capacity is available, and the volunteer can carry the quantity.
Unassigned	Donation D-1008 could not be assigned because no compatible recipient had remaining capacity before the donation expired.
Deferred	Donation D-1014 was considered after D-1002 because D-1002 had less remaining shelf life under the urgency-first greedy rule.

Explainability matters because a user should be able to understand the assumptions, constraints, and priority rule behind a system decision.

6. Required Metrics

Metric	Definition
Donations received	Total number of donation records processed
Donations assigned	Number of donations matched to a recipient and volunteer
Donations unassigned	Number of donations without a feasible match
Food quantity rescued	Total quantity successfully assigned
Food quantity at risk	Quantity left unassigned or expired
Urgent rescue rate	Percentage of earliest-expiring donations assigned
Recipient capacity utilization	Assigned quantity divided by available recipient capacity
Volunteer capacity utilization	Assigned quantity divided by volunteer capacity
FIFO vs. greedy comparison	Difference in assigned donations, rescued quantity, urgent rescue rate, or another justified metric

7. Algorithms and Data Structures

Need	Suggested structure	Reason
Lookup by ID	dict	Expected fast direct retrieval
Detect duplicate IDs	set	Expected fast membership testing
Food-type compatibility	set	Natural representation of accepted categories
FIFO baseline	list or deque	Preserves arrival order
Greedy pending donations	heapq priority queue or sorted list	Selects next donation by explicit priority rule
Assignment history	list	Maintains ordered audit output
Metrics	dict	Stores named aggregate values

Include a short complexity discussion. Define meaningful input variables, such as D for donations, R for recipients, and V for volunteers. Identify the dominant work in your matching procedure and state any assumptions.

8. Testing Requirements

- Normal matching case with compatible donor, recipient, and volunteer.
- Duplicate donation ID.
- Expired or nearly expired donation.
- No compatible recipient food type.
- Insufficient recipient capacity.
- No available volunteer with enough vehicle capacity.
- Two donations competing for the same capacity.
- Empty input files or empty lists.
- A case where FIFO and greedy strategies produce different results.
- A fairness or starvation discussion case: explain which donation may be deferred and why.

Use unittest or pytest. Your tests should verify behavior, not merely run the program.

9. Three-Week Schedule

Week	Milestone	Checkpoint evidence
Week 1	Create repository, data model, validation, and FIFO baseline. Build sample input files and initial tests.	GitHub link; data schema; FIFO output; 5 tests; team-role statement.
Week 2	Implement greedy strategy, explainable decisions, metrics, and FIFO-vs.-greedy comparison.	Greedy demo; comparison report; 10 tests; updated README.
Week 3	Polish code, finalize tests and documentation, add demo materials, and prepare presentation.	Final repository; README; demo video or live demo; final report; contribution statement.

10. Team Responsibilities

Each group has three members. Roles help organize work, but every student must understand the full system and contribute meaningful work.

Role	Primary responsibilities
Algorithm & Optimization Lead	FIFO scheduler, greedy rule, priority selection, complexity analysis, strategy comparison.
Data & Matching Lead	Input files, validation, compatibility checks, capacity rules, data structures, test data.
Product & Quality Lead	Explainable output, metrics, tests, README, demo materials, interface or CLI polish.

All members are responsible for design discussion, code review, debugging, testing, final presentation, and documentation review. Commit history should reflect meaningful contributions from each team member.

11. Final Submission

- GitHub repository link.
- Runnable Python application with clear setup instructions.
- Sample CSV or JSON input files.
- FIFO and greedy scheduling implementations.
- Explainable output for assignments and failures.
- Automated test suite.
- Metrics and FIFO-vs.-greedy comparison report.
- README with architecture, algorithm design, complexity analysis, limitations, and team contributions.
- Three-to-five-minute demo video or live demonstration.
- Individual contribution statement from each student.

12. Portfolio Expectations

Your repository should be something you would be comfortable showing an internship interviewer. A strong submission is not the longest codebase. It is a project that clearly communicates a real problem, a working solution, evidence of testing, thoughtful trade-offs, and professional documentation.

Suggested resume bullet: Built an explainable surplus-food matching and pickup-planning prototype in Python; implemented FIFO and greedy expiry-aware scheduling, compatibility and capacity validation, automated tests, and outcome metrics for a three-person software project.

Suggested LinkedIn description: Built RescueRoute, a surplus-food matching and pickup-planning prototype that compares FIFO and greedy urgency-first strategies for assigning time-sensitive donations to compatible recipient organizations and volunteers. Implemented explainable decisions, capacity constraints, automated tests, and operational metrics in Python.

13. Scope and Academic Integrity

- Complete the minimum viable product before adding dashboards, maps, APIs, optimization libraries, or deployment features.
- Document any external library, tutorial, dataset, or code reference used by the group.
- Do not submit copied code that group members cannot explain.
- Do not claim that the project is clinically validated, food-safety certified, or ready for real operational deployment.
- AI-assisted tools may be used only according to course policy; students remain responsible for understanding, testing, and documenting all submitted code.

Optional stretch features—only after the required system works—include a Streamlit dashboard, simulated map distances, fairness rules, route planning, what-if analysis, or a comparison with a constraint-optimization tool.

Start here: Create your team repository, agree on your data schema and exact greedy rule, and implement a small FIFO baseline that you can test end to end. A simple, correct, documented baseline is the foundation for a strong final project.